Project Plan: Squeezing a Pretrained Nepali Model to 1.58 Bits

5-person team · ~24 weeks · Route D (squeeze while training) · Drafted 2026-10-02 Builds on `claude/route-d-squeeze-while-training.md`. This plan replaces that doc's choice of base model: the main base is now a model that already knows Nepali.

Part 1 — Pretrained Nepali models you can start from

1.1 Shortlist

Model	Paper / source	Size & architecture	Tokenizer	Nepali training data	Weights & licence	Fit for this project
Arkios (base + chat)	Regmi, Pudasaini & Phakami Pun (Karela Technologies), *Arkios: An Open Bilingual English–Nepali LM Trained From Scratch, with a Devanagari-Aware Tokenizer*, arXiv 2608.30092 (2026)	**1.04B**, Llama-style decoder: 18 layers, d=2048, 16 heads / 8 KV heads (GQA), RMSNorm, SwiGLU, RoPE, tied embeddings, 4k context	65,536-token byte-level BPE, Devanagari-aware. **1.69 tokens per Nepali word**	150B tokens total. ~4.18B unique Nepali tokens (Sangraha, IndicCorp V2, Wikipedia + a private set). About 80:1 English:Nepali	**Apache-2.0** on Hugging Face (base + chat). No intermediate checkpoints, optimizer states or training code	⭐ **Primary base**

Kathmandu University GPT-2	Thapa, Nyachhyon, Sharma & Bal, *Development of Pre-Trained Transformer-based Models for the Nepali Language*, arXiv 2411.15734 / CHIPSAL 2025	124M GPT-2 (they also trained BERT and RoBERTa)	BPE, 30,522 or 50,256 vocab	**27.5 GB** from 99 Nepali news sites, the largest Nepali corpus reported	No link in the paper. `mridul3301/nepali-gpt-aug5` on HF is a 0.1B GPT-2 from a co-author's account, but it's undocumented, so confirm with the authors	Pilot / reference
NepaliGPT	*NepaliGPT: A Generative Language Model for the Nepali Language*, arXiv 2506.16399 (2025)	GPT-2, 88.2M	Not documented	9.3 GB "Devanagari Corpus". The paper also releases a **4,296-pair Nepali QA benchmark**	HF `Shushant/thesis_nepaliGPT`, BSD-3-Clause-Clear. Its reported perplexity (26.32) matches the paper's	Pilot. **Its QA benchmark is worth using for evaluation**
Nepali BPE GPT	*Towards Nepali-Language LLMs: Efficient GPT Training with a Nepali BPE Tokenizer*, arXiv	98M decoder, 12 layers	16,384 Nepali-only SentencePiece BPE	10.75 GB cleaned Nepali	Release not confirmed	Reference only

	2512.14585					
LLAMA 3-Nepali 318M	Community model (Aananda-giri), no paper	318M, Llama-3.2-*style* (10 layers, d=1320). These shapes don't match any Meta Llama, so it was effectively trained from scratch	50,006-token BPE	IRIISNEPAL Nepali-Text-Corpus + NepBERTa corpus	HF, raw `.pth` with custom code. Licence not stated	Possible pilot (Llama architecture), but needs format conversion
GPT2-Nepali	Community model (Aananda-giri)	124M GPT-2	Nepali BPE	12 GB NepBERTa text	HF, licence not stated	Pilot only

Adapted multilingual models (useful mostly as references):

Model	Source	What it is	Why not the main base
DAPT-Nepali	Duwal, Prasai & Manandhar (Wiseyak), arXiv 2412.13860	Llama-3-8B adapted to Nepali with 4-bit QLoRA, using 5M synthetic parallel paragraphs	8B is far too big for full QAT on your budget. **Cite its forgetting result**: English MMLU dropped from 0.61 to 0.35
MISHANM Llama-3.2-3B Nepali	Community model	LoRA instruction tune on ~29k Nepali Alpaca examples	Instruction tuning only, so it doesn't make the base model know Nepali any better

Multilingual bases with some Nepali (contrast models):

- **Llama-3.2-1B**: Nepali isn't officially supported, but Hindi is, so it has seen Devanagari. Same size and architecture family as Arkios. ⭐ **Contrast base.**
- **Qwen3-0.6B / 1.7B**: Nepali is in Qwen3's official list of 119 languages. Apache-2.0. ⭐ **Pilot base** (loads natively in HF, quick to iterate).

Optional side track (encoders, classification only): NepBERTa, `IRIISNEPAL/RoBERTa_Nepali_110M`, and the KU RoBERTa (95.60 on Nep-gLUE). The direct precedent for ternary encoders is **TernaryBERT** (Zhang et al., EMNLP 2020). Only add this track if someone has spare capacity.

1.2 Why Arkios is the main base

1. **It is Nepali-native.** It was trained with Nepali from the start, using a tokenizer designed for Devanagari.
2. **Its tokenizer is efficient** (1.69 tokens/word). Ternary weights make each token cheap; an efficient tokenizer means fewer tokens per sentence. The two savings **multiply**, which is the efficiency story nobody has measured.
3. **It is ~1B parameters**, the right size for full QAT on a modest budget.
4. **It uses a Llama-style architecture**, which ternary tooling and `bitnet.cpp` handle most reliably.
5. **Apache-2.0** lets you release your squeezed versions openly.

1.3 Caveats to know before committing

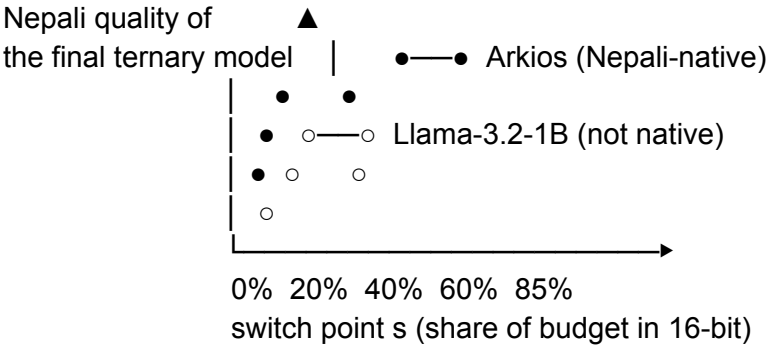
- **No intermediate checkpoints or optimizer states.** You can't switch mid-pretraining exactly the way Nielsen et al. did. You are converting a **fully trained** model, which is the case where the Hugging Face blog found a gradual λ ramp essential.
- **Its Nepali comprehension is weak to start with.** Belebele-Nepali scores are 0.306 (answer-text format) and 0.240 (letter-choice, which is chance). English→Nepali output is in the correct script only 58% of the time. That leaves real room to improve. It also means **multiple-choice benchmarks may sit at the floor and hide differences**, so rely on bits-per-byte and generation-based metrics.
- **Overlap with its own training data.** Arkios already trained on Sangraha, IndicCorpV2 and Wikipedia, so much of your continued-training text will be text it has already seen. That's fine for a squeeze study (Nielsen also kept the data fixed), but **evaluation sets must be checked for contamination**.
- **Repository URL:** the paper says the weights are on HF but doesn't give the exact repo name. Find it in week 1 (search HF for "Arkios" / Karela Technologies) and **check it loads as a standard Llama model in `transformers`.**
- **GPT-2 gotcha (pilot models):** HF's GPT-2 uses `Conv1D` layers, not `nn.Linear`. Tools that swap `nn.Linear` for `BitLinear` will **silently skip them**, so you'd train in full precision without noticing. This is another reason to pilot on Qwen3-0.6B.

Part 2 — What changes in the research question

The original Route D question was what happens when a model learns a new language and gets squeezed at the same time. With a Nepali-native base, the sharper question is:

Does a model that already knows Nepali survive the squeeze better than one that doesn't — and does that change *when* you should switch to 1.58 bits?

That turns the study into a comparison of two curves:



Revised hypotheses

- **H1 (nativeness):** at every switch point, Arkios loses less Nepali quality to the squeeze than Llama-3.2-1B.
- **H2 (where the optimum falls):** Arkios's best switch point sits near Nielsen's 20–40%, because it doesn't need to learn the language. Llama's sits later, because it must first learn Nepali in 16-bit. **The gap between the two optima is the headline finding.**
- **H3 (λ ramp):** both bases are fully trained, so a gradual λ ramp beats an abrupt switch. That would differ from Nielsen's result, consistent with the Hugging Face blog's.
- **H4 (savings multiply):** the end-to-end Nepali CPU speedup of ternary Arkios over FP Llama ≈ (ternary speedup) × (tokenizer advantage). Measure it; don't assume it.
- **H5 (language mix):** shifting the training mix from Arkios's 80:1 English:Nepali towards Nepali during the squeeze raises Nepali quality, and tight ternary capacity makes it cost more English.

Run plan (fixed budget of 2B tokens per run, about 85% Nepali / 15% English replay)

#	Base	Run	Tests
P1–P4	Qwen3-0.6B	Pilot: FP, s=0, s=40%, s=85%, each with 0.5B tokens	Pipeline works; cost per run
A1	Arkios	16-bit only (ceiling)	—
A2	Arkios	16-bit only, then naive rounding to ternary (floor)	—
A3–A7	Arkios	s = 0, 20, 40, 60, 85% (short sigmoid λ ramp)	H1, H2

L1	Llama-3.2-1B	16-bit only (ceiling)	—
L2–L4	Llama-3.2-1B	s = 0, 40, 85%	H1, H2
X1	Arkios	Best s, **abrupt** λ	H3
X2	Arkios	Best s, **optimizer reset** at the switch	Replicates Nielsen
X3	Arkios	Best s, **Nepali-heavy mix** (~95% Nepali)	H5
S1–S4	Arkios + Llama	Nepali instruction tuning of the best ternary and FP models	Nielsen's open question on instruction tuning

Compute estimate (A100 at ~40% utilisation, +20% QAT overhead):

Block	GPU-hours
Pilot (4 runs × ~5 h)	~20
Arkios sweep (7 runs × ~33 h)	~235
Llama contrast (4 runs × ~40 h)	~160
Ablations (3 runs × ~33 h)	~100
Instruction tuning (4 runs × ~3 h)	~15
Total	**~525 → plan for ~630 A100-hours with 20% buffer**

If compute is tight: halve the budget to 1B tokens per run and drop L3 and X2. That's about 300 A100-hours.

Part 3 — The 5-person team

Role	Owns	Backup for
P1 — Research Lead	Experiment design, hypotheses, literature tracking, statistics plan, paper writing, weekly coordination, decision log	P5 (statistics)
P2 — Data Lead	Nepali corpora, cleaning, deduplication, contamination checks, data mixes, tokenised shards, instruction data	P5 (evaluation data)

P3 — Ternary Methods Lead	BitLinear and λ scheduler code, training loop, Arkios runs, loss-spike analysis	P4 (training)
P4 — Infra & Deployment Lead	Compute access, memory/distributed setup, checkpoints, experiment tracking, Llama contrast runs, ternary export, `bitnet.cpp`, CPU/Android benchmarks	P3 (training)
P5 — Evaluation Lead	Evaluation harness, Nepali + English benchmarks, native-authored test set, human evaluation, significance tests	P1 (analysis)

Every person has a backup, so no stage stalls if one person is unavailable.

Part 4 — Stage-by-stage breakdown

Stage 0 — Kick-off and alignment (Weeks 1–2)

Goal: everyone understands the plan; the biggest unknowns are resolved.

Person	Tasks
P1	Run a reading session on 5 core papers: BitNet b1.58, the HF 1.58-bit blog, Nielsen et al., ParetoQ, Arkios. Freeze hypotheses H1–H5. Start the decision log in the project.
P2	Count **actual available clean Nepali tokens** (Sangraha-ne, IndicCorpV2-ne, CC-100/OSCAR-ne, Wikipedia, news, KU's 27.5 GB if obtainable) using the Arkios tokenizer.
P3	Find the Arkios HF repo. Load it in `transformers`, confirm it's a standard Llama layout, and generate some Nepali text. Same for Llama-3.2-1B and Qwen3-0.6B.
P4	**Secure compute** (target ~630 A100-hours: university cluster, research credits or cloud). Set up experiment tracking (e.g. W&B) and the shared storage layout.
P5	Inventory the evaluation sets: NLUE, FLORES-200, Belebele-ne, NepaliGPT's 4,296-pair QA set, Nepali summarisation, English ARC/HellaSwag/PIQA.

Gate G0 (end of week 2): compute is confirmed · Arkios loads · Nepali token count is known (≥2B is ideal; if lower, plan the number of repeat passes) · hypotheses are frozen.

Stage 1 — Foundations (Weeks 2–6)

Goal: a data pipeline, an evaluation harness, and the "Route 0" baseline table with no training yet.

Person	Tasks
P1	Write the statistics plan (seeds, confidence intervals, which differences count as real). Draft the paper's related-work section.
P2	Build the cleaning pipeline: language ID, Unicode normalisation, near-duplicate removal, boilerplate filtering. Produce the 85/15 and 95/5 mixes as tokenised shards. **Remove evaluation sets from the training data.** Collect Nepali instruction data (IndicAlign, UPDESH, Aya).
P3	Implement `BitLinear` + the λ scheduler (or wrap `onebitllms`). Unit tests: λ=0 must reproduce the FP model's outputs exactly; λ=1 must give only −1/0/+1 weights.
P4	Training launcher with resume, checkpointing every N steps, logging of loss, λ and gradient norm. Measure memory and throughput for 1B models on your hardware. Test the **ternary export → `bitnet.cpp`** path with a dummy model.
P5	Build the evaluation harness (lm-eval-harness + custom Nepali tasks + **bits-per-byte**). **Reproduce Arkios's published numbers** (ARC-E 0.735 10-shot, Belebele) as a sanity check. Start writing the native-authored Nepali test set (target 200–300 items).
All	**Baseline table:** Arkios, Llama-3.2-1B, Qwen3 at FP16 / 8-bit / 4-bit, plus tokens per Nepali word and CPU speed.

Gate G1 (end of week 6): the harness reproduces Arkios's numbers within tolerance · data shards are ready · BitLinear unit tests pass · export to `bitnet.cpp` works · the baseline table is done. **The baseline table is the first internal result.**

Stage 2 — Pilot (Weeks 6–8)

Goal: prove the full pipeline end to end on a cheap model before spending real compute.

Person	Tasks
P3	Run pilots P1–P4 on Qwen3-0.6B with 0.5B tokens. Check that the loss spike appears at the switch and recovers.

P4	Measure real GPU-hours per run → **finalise the run count**. Export a pilot ternary model and run it in `bitnet.cpp`.
P5	Evaluate the pilot checkpoints. Check the metrics are **sensitive enough** to separate the runs (if Belebele is stuck at chance, drop it as a primary metric).
P2	Fix any data problems the pilot exposes (e.g. odd generations caused by noisy sources).
P1	Review the pilot. Decide the λ ramp length and the final switch-point grid.

Gate G2 (end of week 8), go / no-go: ternary training is stable · the exported model runs on CPU · the metrics can tell runs apart · cost per run fits the budget. If something fails, fix it here; the next stage spends ~75% of the compute.

Stage 3 — Main sweep on Arkios (Weeks 9–14)

Goal: the Arkios curve, runs A1–A7.

Person	Tasks
P3	Run A1–A7. Analyse the loss spikes (height, recovery steps) per switch point.
P4	Keep the GPUs busy (queue runs back to back). Manage checkpoint storage. Export every final model.
P5	Evaluate each checkpoint as soon as it lands: Nepali bits-per-byte, NLUE subset, FLORES chrF++, NepaliGPT-QA, English retention.
P2	Prepare the Nepali-heavy mix for X3. Prepare the instruction-tuning set (cleaned, formatted, deduplicated against the evaluation sets).
P1	Keep the results tracker updated. **Mid-stage check (week 11):** if the curve peaks between two tested points, add a point there. Draft the methods section.

Gate G3 (week 11): the curve shape makes sense. **Gate at week 14:** the Arkios sweep is complete.

Stage 4 — Contrast base and ablations (Weeks 12–17, overlapping Stage 3)

Goal: the Llama curve (L1–L4) and the λ / optimizer / mix ablations (X1–X3).

Person	Tasks
P4	Run L1–L4 on Llama-3.2-1B.
P3	Run X1–X3 at Arkios's best switch point.
P5	Evaluate everything. Run **significance tests** (bootstrap confidence intervals over evaluation items; several seeds on at least the best and worst runs).
P1	Build the main two-curve figure. Test H1–H5 and write down which held and which didn't.
P2	Measure tokens per word and **effective Nepali throughput** across all final models (supports H4).

Gate G4 (end of week 17): all pretraining-style runs are done · **results frozen** (no new runs after this point without P1's sign-off).

Stage 5 — Instruction tuning and deployment (Weeks 16–19)

Goal: usable Nepali assistants, and real-world efficiency numbers.

Person	Tasks
P3	Instruction-tune (S1–S4) the best ternary and FP versions of both bases, all on the same data.
P4	Benchmark on CPU and Android with `bitnet.cpp`: latency per Nepali sentence, peak memory, energy if measurable. Build a small demo (CLI or simple chat UI).
P5	Human evaluation: native speakers rate fluency, correctness and script fidelity on the native-authored set, blind to which model produced each answer.
P2	Help with the human evaluation (annotator guidelines, agreement checks).
P1	Answer Nielsen's open question: does instruction tuning help ternary models less than FP ones?

Stage 6 — Analysis and writing (Weeks 18–22)

Person	Tasks

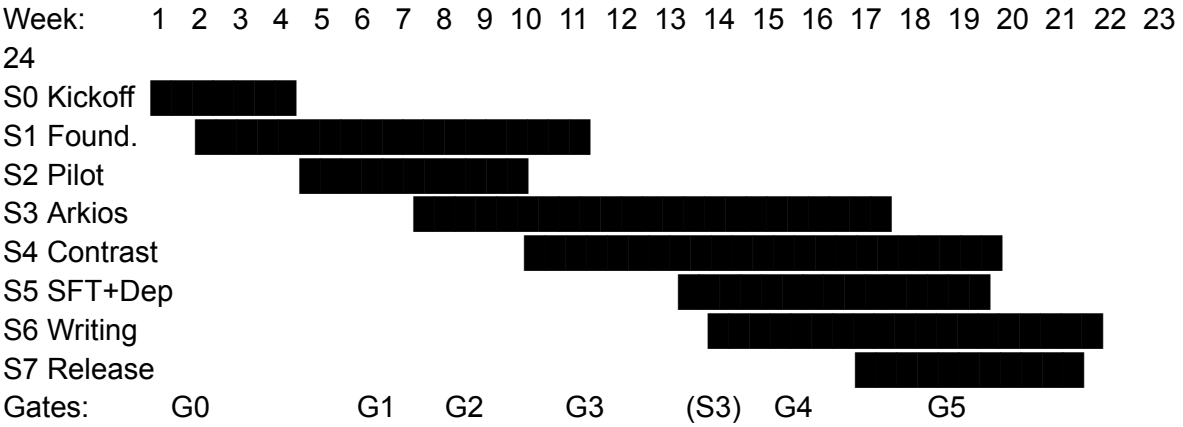
P1	Paper lead: introduction, framing, discussion, limitations. Integrates everyone's sections.
P3	Methods section (BitLinear, λ schedule, training details), loss-spike analysis.
P4	Efficiency and deployment section, hardware table, reproducibility appendix.
P5	Evaluation section, significance tables, human-evaluation analysis.
P2	Data section and **datasheet** (sources, cleaning, contamination checks, licences).

Gate G5 (week 21): full internal draft. Each person reviews one section they didn't write.

Stage 7 — Release and submission (Weeks 22–24)

Person	Tasks
P4	Publish the ternary checkpoints (Apache-2.0, inherited from Arkios; Llama-derived models follow the Llama licence) and `bitnet.cpp`-ready files.
P3	Clean up and release the training code with configs for every run.
P5	Release the evaluation harness and the native-authored test set.
P2	Release data recipes and scripts (not the raw scraped text, if licensing is unclear).
P1	Choose the venue and submit (ACL-family workshops on South Asian / low-resource languages, e.g. CHIPSAL, or efficiency workshops). arXiv preprint.

Part 5 — Timeline at a glance



Each person's busiest stretch: P2 in weeks 1–8 (data) · P3 in weeks 3–17 (training) · P4 in weeks 1–2 and 9–19 (compute, runs, deployment) · P5 throughout, peaking in weeks 9–19 · P1 at the start (design) and end (writing).

Part 6 — How the team works

- **Weekly 45-minute sync.** Each person covers what's done, what's blocking them, and what's next. P1 keeps the decision log as a project doc.
- **Run registry.** Every run has an ID (`A4-arkios-s40-seed1`), config file, W&B link, checkpoint path and evaluation row. Nothing is "the good run from Tuesday".
- **Results freeze at G4.** This prevents endless new experiments late in the project.
- **Two-person rule for training code.** P3 and P4 review each other's changes before any expensive run.
- **Risk owners:**

Risk	Owner	Early warning	Fallback
Not enough clean Nepali tokens	P2	G0 token count	1B tokens per run, or allow a 2nd pass over the data
Compute falls through	P4	G0	The ~300 GPU-hour reduced plan
Arkios won't load or convert cleanly	P3	Week 1	Qwen3-1.7B as the Nepali-capable main base
Metrics can't tell runs apart	P5	G2	Bits-per-byte + generation metrics + human eval as primary
`bitnet.cpp` doesn't support the architecture	P4	G1	Report efficiency from a reference ternary kernel; flag it as a limitation
Scooped by a multilingual 1.58-bit paper	P1	Monthly arXiv check	The Nepali-native vs non-native comparison and the Nepali evaluation set remain unique

---

Sources

- Arkios (arXiv 2608.30092)
- Development of Pre-Trained Transformer-based Models for Nepali — Kathmandu University (arXiv 2411.15734) · ACL Anthology
- NepaliGPT (arXiv 2506.16399) · HF: Shushant/thesis_nepaliGPT
- Towards Nepali-Language LLMs: Nepali BPE Tokenizer (arXiv 2512.14585)

- HF: Aananda-giri/LLAMA3-Nepali · HF: Aananda-giri/GPT2-Nepali
- HF: mridul3301/nepali-gpt-aug5
- Domain-adaptive Continual Learning for Nepali — Wiseyak (arXiv 2412.13860)
- HF: MISHANM Llama-3.2-3B Nepali
- Qwen3 blog (supported languages)
- HF: IRIISNEPAL/RoBERTa_Nepali_110M
- NLUE benchmark (arXiv 2411.19244)
- Nielsen et al., Continual QAT Pre-Training (arXiv 2502.11895)
- HF blog: Fine-tuning LLMs to 1.58bit
- onebitllms · bitnet.cpp